

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего профессионального образования

"МИРЭА — Российский технологический университет"

РТУ МИРЭА

Институт технологий управления (ИТУ)
Кафедра информационных технологий в государственном управлении (ИТГУ)

ОТЧЕТ ПО ПРАКТИЧЕСКИМ РАБОТАМ

по дисциплине
«Объектно-ориентированный анализ и программирование»

на тему «Практическая работа 1-4»

Студенты группы УИБО-02-24 Шульгин.М.Д. _____
(учебная группа, фамилия, имя, отчество студента) *(подпись студента)*

Преподаватель Преподаватель Зорина.Н.В. _____
(должность, ученая степень, звание, фамилия, имя, отчество преподавателя) *(подпись преподавателя)*

Работа выполнена «24» октября 2026 г.

«Зачтено» «24» октября 2026 г.

Москва 2026

СОДЕРЖАНИЕ

1.Практическая работа №1 3
2.Практическая работа №2 7
3.Практическая работа №3 9
4.Практическая работа №4 13
ЗАКЛЮЧЕНИЕ 17
ПРИЛОЖЕНИЕ А 18
ПРИЛОЖЕНИЕ Б 20
ПРИЛОЖЕНИЕ В 21
ПРИЛОЖЕНИЕ Г 26

1.Практическая работа №1

В задании 3 для массива, заданного в коде, вычисляются сумма элементов в цикле for и среднее арифметическое (рисунок 1).

```
6           // Задание 3
7           int[] a = {1, 2, 3, 4, 5};
8           int sum = 0;
9           for (int i = 0; i < a.length; i++) {
10              sum += a[i];
11          }
12          System.out.println("Сумма:");
13          System.out.println(sum);
14          System.out.println("Среднее:");
15          System.out.println((double) sum / a.length);
```

Рисунок 1 – Сумма и среднее арифметическое элементов массива

В задании 4 массив задаётся в коде. Сумма элементов вычисляется двумя способами: циклом do while и циклом while. В цикле for определяются минимальный и максимальный элементы (рисунок 2).

```java
        // Задание 4
        int[] b = {4, -2, 7, 10, 1};

        int n = b.length;

        int sum1 = 0;
        int i = 0;
        do {
            sum1 += b[i];
            i++;
        } while (i < n);

        int sum2 = 0;
        i = 0;
        while (i < n) {
            sum2 += b[i];
            i++;
        }

        int min = b[0];
        int max = b[0];
        for (i = 0; i < n; i++) {
            if (b[i] < min) {
                min = b[i];
            }
            if (b[i] > max) {
                max = b[i];
            }
        }

        System.out.println("Сумма do while:");
        System.out.println(sum1);
        System.out.println("Сумма while:");
        System.out.println(sum2);
        System.out.println("Минимум:");
        System.out.println(min);
        System.out.println("Максимум:");
        System.out.println(max);
```

Рисунок 2 – Вычисление суммы, минимума и максимума

В задании 5 выводятся аргументы командной строки, в задании 6 – первые 10 чисел гармонического ряда, в задании 7 вычисляется факториал числа (рисунок 3). Факториал вычисляется в отдельном статическом методе factorial (рисунок 4).

```
56          // Задание 5
57          System.out.println("Аргументы программы:");
58          for (i = 0; i < args.length; i++) {
59              System.out.println(args[i]);
60          }
61
62          // Задание 6
63          System.out.println("Гармонический ряд:");
64          for (i = 1; i <= 10; i++) {
65              System.out.println(1.0 / i);
66          }
67
68          // Задание 7
69          System.out.println("Факториал числа 5:");
70          System.out.println(factorial(5));
```

Рисунок 3 – Аргументы командной строки, гармонический ряд и факториал

```
73      public static long factorial(int n) {
74          long result = 1;
75          for (int i = 1; i <= n; i++) {
76              result *= i;
77          }
78          return result;
79      }
```

Рисунок 4 – Метод вычисления факториала

Результат работы программы при запуске с аргументами hello java приведён на рисунке 5. Полный листинг программы приведён в приложении А.

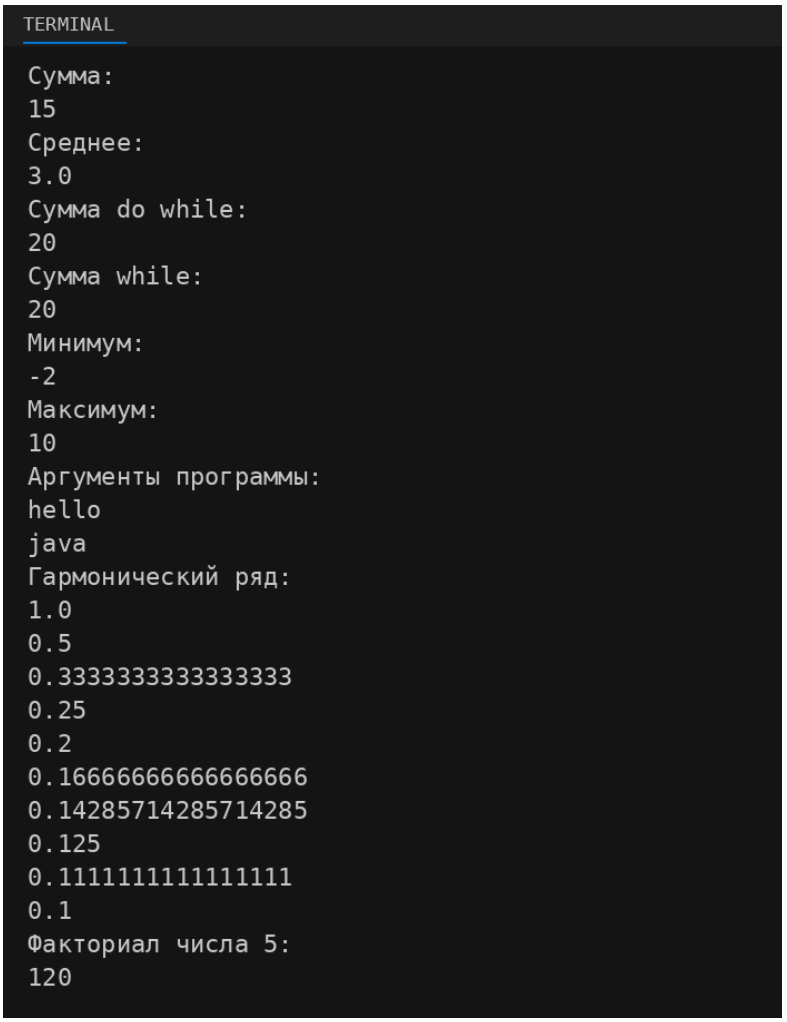


Рисунок 5 – Результат выполнения практической работы №1

2.Практическая работа №2

Создан класс Author с закрытыми полями name, email и gender, конструктором, методами get и set и переопределённым методом toString (рисунок 6).

```java
package ru.mirea;

public class Author {
    private String name;
    private String email;
    private char gender;

    public Author(String name, String email, char gender) {
        this.name = name;
        this.email = email;
        this.gender = gender;
    }

    public String getName() { return name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public char getGender() { return gender; }

    @Override
    public String toString() {
        return name + " (" + gender + ") at " + email;
    }
}
```

Рисунок 6 – Класс Author

В классе Main создаётся объект автора, изменяется адрес электронной почты и выводятся значения полей (рисунок 7). Результат работы программы приведён на рисунке 8, полный листинг – в приложении Б.

```java
package ru.mirea;

public class Main {
    public static void main(String[] args) {
        Author author = new Author("Иван", "ivan@mail.ru", 'M');
        System.out.println(author);
        author.setEmail("new@mail.ru");
        System.out.println(author.getName());
        System.out.println(author.getEmail());
        System.out.println(author.getGender());
        System.out.println(author);
    }
}
```

Рисунок 7 – Класс Main

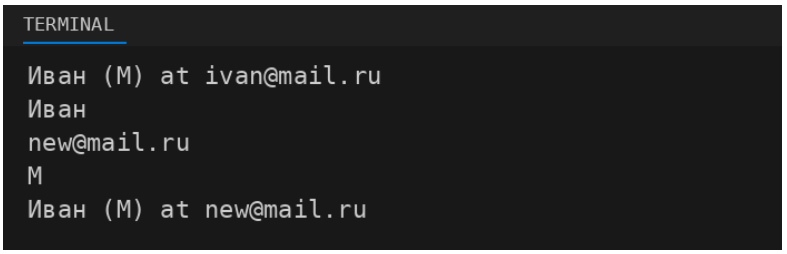


Рисунок 8 – Результат выполнения практической работы №2

3.Практическая работа №3

Создан абстрактный класс Shape с полями color и filled и абстрактными методами getArea и getPerimeter (рисунок 9). От него наследуются классы Circle и Rectangle, а от Rectangle – класс Square, в котором методы setWidth и setLength переопределены так, чтобы стороны квадрата оставались равными (рисунок 10).

```java
package ru.mirea;

public abstract class Shape {
    protected String color;
    protected boolean filled;

    public Shape() {
        color = "red";
        filled = true;
    }

    public Shape(String color, boolean filled) {
        this.color = color;
        this.filled = filled;
    }

    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }
    public boolean isFilled() { return filled; }
    public void setFilled(boolean filled) { this.filled = filled; }

    public abstract double getArea();
    public abstract double getPerimeter();

    public String toString() {
        return "color=" + color + ", filled=" + filled;
```

Рисунок 9 – Абстрактный класс Shape

```java
package ru.mirea;

public class Square extends Rectangle {
    public Square() {
        super();
    }

    public Square(double side) {
        super(side, side);
    }

    public Square(double side, String color, boolean filled) {
        super(side, side, color, filled);
    }

    public double getSide() { return width; }

    public void setSide(double side) {
        width = side;
        length = side;
    }

    public void setWidth(double side) { setSide(side); }
    public void setLength(double side) { setSide(side); }

    public String toString() {
        return "Square: side=" + width + ", " + super.toString();
```

Рисунок 10 – Класс Square

В методе main проверяется восходящее и нисходящее приведение типов. Строки из задания, которые приводят к ошибке компиляции, оставлены в комментариях с указанием причины (рисунок 11).

```java
package ru.mirea;

public class Main {
    public static void main(String[] args) {
        // Задание 1. Фигуры
        Shape s1 = new Circle(5.5, "RED", false);
        System.out.println(s1);
        System.out.println(s1.getArea());
        System.out.println(s1.getPerimeter());
        Circle c1 = (Circle) s1;
        System.out.println(c1.getRadius());

        Shape s3 = new Rectangle(1, 2, "RED", false);
        System.out.println(s3);
        System.out.println(s3.getArea());
        Rectangle r1 = (Rectangle) s3;
        System.out.println(r1.getLength());

        Shape s4 = new Square(6.6);
        System.out.println(s4);
        System.out.println(s4.getArea());
        Rectangle r2 = (Rectangle) s4;
        Square sq1 = (Square) r2;
        System.out.println(sq1.getSide());

        // Ошибочные строки из задания оставлены в комментариях:
        // s1.getRadius();       // у Shape нет getRadius()
        // Shape s2 = new Shape(); // Shape - абстрактный класс
        // s3.getLength();       // у Shape нет getLength()
        // s4.getSide();         // у Shape нет getSide()
        // r2.getSide();         // у Rectangle нет getSide()
```

Рисунок 11 – Проверка приведения типов в методе main

Во втором задании создан интерфейс Movable с методами перемещения. Его реализуют классы MovablePoint, MovableCircle и MovableRectangle. Прямоугольник задаётся двумя точками MovablePoint, а метод sameSpeed проверяет, что скорости точек совпадают (рисунок 12).

```java
package ru.mirea;

public class MovableRectangle implements Movable {
    private MovablePoint topLeft;
    private MovablePoint bottomRight;

    public MovableRectangle(int x1, int y1, int x2, int y2, int xSpeed, int ySpeed) {
        topLeft = new MovablePoint(x1, y1, xSpeed, ySpeed);
        bottomRight = new MovablePoint(x2, y2, xSpeed, ySpeed);
    }

    public boolean sameSpeed() {
        return topLeft.xSpeed == bottomRight.xSpeed && topLeft.ySpeed == bottomRight.ySpeed;
    }

    public void moveUp() {
        topLeft.moveUp();
        bottomRight.moveUp();
    }

    public void moveDown() {
        topLeft.moveDown();
        bottomRight.moveDown();
    }
```

Рисунок 12 – Класс MovableRectangle

Результат работы программы приведён на рисунке 13, полный листинг – в приложении В.

```
TERMINAL
Circle: radius=5.5, color=RED, filled=false
95.03317777109123
34.55751918948772
5.5
Rectangle: width=1.0, length=2.0, color=RED, filled=false
2.0
2.0
Square: side=6.6, Rectangle: width=6.6, length=6.6, color=red, filled=true
43.559999999999995
6.6
Point: x=0, y=3, xSpeed=2, ySpeed=3
MovableCircle: radius=5, Point: x=3, y=1, xSpeed=2, ySpeed=2
true
MovableRectangle: Point: x=0, y=3, xSpeed=1, ySpeed=1; Point: x=6, y=-1, xSpeed=1, ySpeed=1
```

Рисунок 13 – Результат выполнения практической работы №3

4.Практическая работа №4

Создано перечисление Season с полями названия и средней температуры, конструктором и методами get. Метод getDescription возвращает «Холодное время года», а для константы SUMMER он переопределён (рисунок 14).

```
1  package ru.mirea;
2
3  public enum Season {
4      WINTER("Зима", -12),
5      SPRING("Весна", 9),
6      SUMMER("Лето", 24) {
7          @Override
8          public String getDescription() {
9              return "Теплое время года";
10         }
11     },
12     AUTUMN("Осень", 7);
13
14     private String name;
15     private int temperature;
16
17     Season(String name, int temperature) {
18         this.name = name;
19         this.temperature = temperature;
20     }
21
22     public String getName() { return name; }
23     public int getTemperature() { return temperature; }
24     public String getDescription() { return "Холодное время года"; }
25 }
```

Рисунок 14 – Перечисление Season

Метод printSeason с помощью оператора switch выводит описание для переданного времени года (рисунок 15). В методе main выводится любимое время года, а затем в цикле перебираются все константы перечисления (рисунок 16).

```java
    public static void printSeason(Season season) {
        switch (season) {
            case WINTER:
                System.out.println("Зимой нравится снег и морозная погода.");
                break;
            case SPRING:
                System.out.println("Весной становится теплее и всё зеленеет.");
                break;
            case SUMMER:
                System.out.println("Летом можно долго гулять вечером.");
                break;
            case AUTUMN:
                System.out.println("Осенью нравится прохлада и разноцветные листья.");
                break;
        }
    }
```

Рисунок 15 – Метод printSeason

```
TERMINAL
Времена года
Мое любимое время года:
Осень
Средняя температура:
7
Осенью нравится прохлада и разноцветные листья.

Зима
Температура:
-12
Холодное время года

Весна
Температура:
9
Холодное время года

Лето
Температура:
24
Теплое время года

Осень
Температура:
7
Холодное время года
```

Рисунок 16 – Результат работы с перечислением Season

Для задания «Ателье» создано перечисление ClothingSize с европейским размером, интерфейсы MenClothing и WomenClothing и абстрактный класс Clothes (рисунок 17). Классы TShirt и Pants реализуют оба интерфейса (рисунок 18), Skirt – только WomenClothing, Tie – только MenClothing.

```java
package ru.mirea;

public abstract class Clothes {
    protected ClothingSize size;
    protected double price;
    protected String color;
    protected String brand;

    public Clothes(ClothingSize size, double price, String color) {
        this.size = size;
        this.price = price;
        this.color = color;
    }

    public void setBrand(String brand) {
        this.brand = brand;
    }

    @Override
    public String toString() {
        return brand + ", размер " + size + ", цвет " + color
                + ", цена " + (int) price + " руб.";
    }
}
```

Рисунок 17 – Абстрактный класс Clothes

```java
package ru.mirea;

public class TShirt extends Clothes implements MenClothing, WomenClothing {
    public TShirt(ClothingSize size, double price, String color) {
        super(size, price, color);
    }

    @Override
    public void dressMan() {
        System.out.println("Мужская футболка:");
        System.out.println(this);
    }

    @Override
    public void dressWomen() {
        System.out.println("Женская футболка:");
        System.out.println(this);
    }
}
```

Рисунок 18 – Класс TShirt

Класс Atelier содержит методы dressMan и dressWomen, которые проверяют через instanceof, реализует ли элемент массива нужный интерфейс, и выводят только подходящую одежду (рисунок 19). Цены задаются случайным образом через класс Random. Результат приведён на рисунке 20, полный листинг – в приложении Г.

```java
package ru.mirea;

public class Atelier {
    public void dressMan(Clothes[] clothes) {
        for (int i = 0; i < clothes.length; i++) {
            if (clothes[i] instanceof MenClothing) {
                MenClothing item = (MenClothing) clothes[i];
                item.dressMan();
            }
        }
    }

    public void dressWomen(Clothes[] clothes) {
        for (int i = 0; i < clothes.length; i++) {
            if (clothes[i] instanceof WomenClothing) {
                WomenClothing item = (WomenClothing) clothes[i];
                item.dressWomen();
            }
        }
    }
}
```

Рисунок 19 – Класс Atelier

```
TERMINAL
Ателье
Мужская одежда:
Мужская футболка:
TBOE, размер M, цвет Белый, цена 1293 руб.
Мужские штаны:
Gloria Jeans, размер L, цвет Черный, цена 2887 руб.
Галстук:
O'STIN, размер M, цвет Синий, цена 1046 руб.
Женская одежда:
Женская футболка:
TBOE, размер M, цвет Белый, цена 1293 руб.
Женские штаны:
Gloria Jeans, размер L, цвет Черный, цена 2887 руб.
Юбка:
ZARINA, размер S, цвет Бежевый, цена 2434 руб.
```

Рисунок 20 – Результат работы программы «Ателье»

ЗАКЛЮЧЕНИЕ

В ходе выполнения практических работ №1–4 были изучены основы языка Java: работа с массивами, циклы for, while и do while, аргументы командной строки и статические методы. Освоено создание классов с закрытыми полями, конструкторами, методами get и set и методом toString. Рассмотрены наследование, абстрактные классы, переопределение методов, восходящее и нисходящее приведение типов, а также интерфейсы. Изучены перечисления с полями, конструкторами и переопределением методов для отдельных констант, оператор switch и проверка типа через instanceof. Все программы скомпилированы и работают корректно.

ПРИЛОЖЕНИЕ А

Листинг практической работы №1

Файл Main.java

```
package ru.mirea;


public class Main {
    public static void main(String[] args) {
        // Задание 3
        int[] a = {1, 2, 3, 4, 5};
        int sum = 0;
        for (int i = 0; i < a.length; i++) {
            sum += a[i];
        }
        System.out.println("Сумма:");
        System.out.println(sum);
        System.out.println("Среднее:");
        System.out.println((double) sum / a.length);

        // Задание 4
        int[] b = {4, -2, 7, 10, 1};

        int n = b.length;

        int sum1 = 0;
        int i = 0;
        do {
            sum1 += b[i];
            i++;
        } while (i < n);

        int sum2 = 0;
        i = 0;
        while (i < n) {
            sum2 += b[i];
            i++;
        }

        int min = b[0];
        int max = b[0];
        for (i = 0; i < n; i++) {
            if (b[i] < min) {
                min = b[i];
            }
            if (b[i] > max) {
                max = b[i];
            }
        }

        System.out.println("Сумма do while:");
        System.out.println(sum1);
        System.out.println("Сумма while:");
        System.out.println(sum2);
        System.out.println("Минимум:");
        System.out.println(min);
        System.out.println("Максимум:");
        System.out.println(max);

        // Задание 5
```

```
        System.out.println("Аргументы программы:");
        for (i = 0; i < args.length; i++) {
            System.out.println(args[i]);
        }

        // Задание 6
        System.out.println("Гармонический ряд:");
        for (i = 1; i <= 10; i++) {
            System.out.println(1.0 / i);
        }

        // Задание 7
        System.out.println("Факториал числа 5:");
        System.out.println(factorial(5));
    }

    public static long factorial(int n) {
        long result = 1;
        for (int i = 1; i <= n; i++) {
            result *= i;
        }
        return result;
    }
}
```

ПРИЛОЖЕНИЕ Б

Листинг практической работы №2

Файл Main.java

```
package ru.mirea;

public class Main {
    public static void main(String[] args) {
        Author author = new Author("Иван", "ivan@mail.ru", 'M');
        System.out.println(author);
        author.setEmail("new@mail.ru");
        System.out.println(author.getName());
        System.out.println(author.getEmail());
        System.out.println(author.getGender());
        System.out.println(author);
    }
}
```

Файл Author.java

```
package ru.mirea;

public class Author {
    private String name;
    private String email;
    private char gender;

    public Author(String name, String email, char gender) {
        this.name = name;
        this.email = email;
        this.gender = gender;
    }

    public String getName() { return name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public char getGender() { return gender; }

    @Override
    public String toString() {
        return name + " (" + gender + ") at " + email;
    }
}
```

ПРИЛОЖЕНИЕ В

Листинг практической работы №3

Файл Main.java

```
package ru.mirea;

public class Main {
    public static void main(String[] args) {
        // Задание 1. Фигуры
        Shape s1 = new Circle(5.5, "RED", false);
        System.out.println(s1);
        System.out.println(s1.getArea());
        System.out.println(s1.getPerimeter());
        Circle c1 = (Circle) s1;
        System.out.println(c1.getRadius());

        Shape s3 = new Rectangle(1, 2, "RED", false);
        System.out.println(s3);
        System.out.println(s3.getArea());
        Rectangle r1 = (Rectangle) s3;
        System.out.println(r1.getLength());

        Shape s4 = new Square(6.6);
        System.out.println(s4);
        System.out.println(s4.getArea());
        Rectangle r2 = (Rectangle) s4;
        Square sq1 = (Square) r2;
        System.out.println(sq1.getSide());

        // Ошибочные строки из задания оставлены в комментариях:
        // s1.getRadius();       // у Shape нет getRadius()
        // Shape s2 = new Shape(); // Shape - абстрактный класс
        // s3.getLength();       // у Shape нет getLength()
        // s4.getSide();         // у Shape нет getSide()
        // r2.getSide();         // у Rectangle нет getSide()

        // Задание 2. Движение
        MovablePoint p = new MovablePoint(0, 0, 2, 3);
        p.moveUp();
        System.out.println(p);

        MovableCircle mc = new MovableCircle(1, 1, 2, 2, 5);
        mc.moveRight();
        System.out.println(mc);

        MovableRectangle mr = new MovableRectangle(0, 4, 6, 0, 1, 1);
        System.out.println(mr.sameSpeed());
        mr.moveDown();
        System.out.println(mr);
    }
}
```

Файл Shape.java

```
package ru.mirea;

public abstract class Shape {
    protected String color;
    protected boolean filled;
```

```
    public Shape() {
        color = "red";
        filled = true;
    }

    public Shape(String color, boolean filled) {
        this.color = color;
        this.filled = filled;
    }

    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }
    public boolean isFilled() { return filled; }
    public void setFilled(boolean filled) { this.filled = filled; }

    public abstract double getArea();
    public abstract double getPerimeter();

    public String toString() {
        return "color=" + color + ", filled=" + filled;
    }
}
```

Файл Circle.java

```
package ru.mirea;

public class Circle extends Shape {
    protected double radius;

    public Circle() {
        super();
        radius = 1;
    }

    public Circle(double radius) {
        super();
        this.radius = radius;
    }

    public Circle(double radius, String color, boolean filled) {
        super(color, filled);
        this.radius = radius;
    }

    public double getRadius() { return radius; }
    public void setRadius(double radius) { this.radius = radius; }

    public double getArea() {
        return Math.PI * radius * radius;
    }

    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }

    public String toString() {
        return "Circle: radius=" + radius + ", " + super.toString();
    }
}
```

Файл Rectangle.java

```
package ru.mirea;

public class Rectangle extends Shape {
    protected double width;
    protected double length;

    public Rectangle() {
        super();
        width = 1;
        length = 1;
    }

    public Rectangle(double width, double length) {
        super();
        this.width = width;
        this.length = length;
    }

    public Rectangle(double width, double length, String color, boolean filled)
{
        super(color, filled);
        this.width = width;
        this.length = length;
    }

    public double getWidth() { return width; }
    public void setWidth(double width) { this.width = width; }
    public double getLength() { return length; }
    public void setLength(double length) { this.length = length; }

    public double getArea() {
        return width * length;
    }

    public double getPerimeter() {
        return 2 * (width + length);
    }

    public String toString() {
        return "Rectangle: width=" + width + ", length=" + length + ", " +
super.toString();
    }
}
```

Файл Square.java

```
package ru.mirea;

public class Square extends Rectangle {
    public Square() {
        super();
    }

    public Square(double side) {
        super(side, side);
    }

    public Square(double side, String color, boolean filled) {
        super(side, side, color, filled);
    }
```

```
    public double getSide() { return width; }

    public void setSide(double side) {
        width = side;
        length = side;
    }

    public void setWidth(double side) { setSide(side); }
    public void setLength(double side) { setSide(side); }

    public String toString() {
        return "Square: side=" + width + ", " + super.toString();
    }
}
```

Файл Movable.java

```
package ru.mirea;

public interface Movable {
    void moveUp();
    void moveDown();
    void moveLeft();
    void moveRight();
}
```

Файл MovablePoint.java

```
package ru.mirea;

public class MovablePoint implements Movable {
    int x, y, xSpeed, ySpeed;

    public MovablePoint(int x, int y, int xSpeed, int ySpeed) {
        this.x = x;
        this.y = y;
        this.xSpeed = xSpeed;
        this.ySpeed = ySpeed;
    }

    public void moveUp() { y = y + ySpeed; }
    public void moveDown() { y = y - ySpeed; }
    public void moveLeft() { x = x - xSpeed; }
    public void moveRight() { x = x + xSpeed; }

    public String toString() {
        return "Point: x=" + x + ", y=" + y + ", xSpeed=" + xSpeed + ",
ySpeed=" + ySpeed;
    }
}
```

Файл MovableCircle.java

```
package ru.mirea;

public class MovableCircle implements Movable {
    private int radius;
    private MovablePoint center;

    public MovableCircle(int x, int y, int xSpeed, int ySpeed, int radius) {
        this.radius = radius;
```

```
        center = new MovablePoint(x, y, xSpeed, ySpeed);
    }

    public void moveUp() { center.moveUp(); }
    public void moveDown() { center.moveDown(); }
    public void moveLeft() { center.moveLeft(); }
    public void moveRight() { center.moveRight(); }

    public String toString() {
        return "MovableCircle: radius=" + radius + ", " + center;
    }
}
```

Файл MovableRectangle.java

```
package ru.mirea;

public class MovableRectangle implements Movable {
    private MovablePoint topLeft;
    private MovablePoint bottomRight;

    public MovableRectangle(int x1, int y1, int x2, int y2, int xSpeed, int
ySpeed) {
        topLeft = new MovablePoint(x1, y1, xSpeed, ySpeed);
        bottomRight = new MovablePoint(x2, y2, xSpeed, ySpeed);
    }

    public boolean sameSpeed() {
        return topLeft.xSpeed == bottomRight.xSpeed && topLeft.ySpeed ==
bottomRight.ySpeed;
    }

    public void moveUp() {
        topLeft.moveUp();
        bottomRight.moveUp();
    }

    public void moveDown() {
        topLeft.moveDown();
        bottomRight.moveDown();
    }

    public void moveLeft() {
        topLeft.moveLeft();
        bottomRight.moveLeft();
    }

    public void moveRight() {
        topLeft.moveRight();
        bottomRight.moveRight();
    }

    public String toString() {
        return "MovableRectangle: " + topLeft + "; " + bottomRight;
    }
}
```

ПРИЛОЖЕНИЕ Г

Листинг практической работы №4

Файл Main.java

```
package ru.mirea;

import java.util.Random;

public class Main {
    public static void main(String[] args) {
        System.out.println("Времена года");
        Season favorite = Season.AUTUMN;
        System.out.println("Мое любимое время года:");
        System.out.println(favorite.getName());
        System.out.println("Средняя температура:");
        System.out.println(favorite.getTemperature());
        printSeason(favorite);

        for (Season season : Season.values()) {
            System.out.println();
            System.out.println(season.getName());
            System.out.println("Температура:");
            System.out.println(season.getTemperature());
            System.out.println(season.getDescription());
        }

        System.out.println();
        System.out.println("Ателье");
        Random random = new Random();
        Clothes[] clothes = {
                new TShirt(ClothingSize.M, 600 + random.nextInt(900), "Белый"),
                new Pants(ClothingSize.L, 1800 + random.nextInt(1700),
"Черный"),
                new Skirt(ClothingSize.S, 1300 + random.nextInt(1400),
"Бежевый"),
                new Tie(ClothingSize.M, 400 + random.nextInt(700), "Синий")
        };
        clothes[0].setBrand("TBOE");
        clothes[1].setBrand("Gloria Jeans");
        clothes[2].setBrand("ZARINA");
        clothes[3].setBrand("O'STIN");

        Atelier atelier = new Atelier();
        System.out.println("Мужская одежда:");
        atelier.dressMan(clothes);
        System.out.println("Женская одежда:");
        atelier.dressWomen(clothes);
    }

    public static void printSeason(Season season) {
        switch (season) {
            case WINTER:
                System.out.println("Зимой нравится снег и морозная погода.");
                break;
            case SPRING:
                System.out.println("Весной становится теплее и всё зеленеет.");
                break;
            case SUMMER:
                System.out.println("Летом можно долго гулять вечером.");
                break;
```

```
            case AUTUMN:
                System.out.println("Осенью нравится прохлада и разноцветные
листья.");
                break;
        }
    }
}
```

Файл Season.java

```
package ru.mirea;

public enum Season {
    WINTER("Зима", -12),
    SPRING("Весна", 9),
    SUMMER("Лето", 24) {
        @Override
        public String getDescription() {
            return "Теплое время года";
        }
    },
    AUTUMN("Осень", 7);

    private String name;
    private int temperature;

    Season(String name, int temperature) {
        this.name = name;
        this.temperature = temperature;
    }

    public String getName() { return name; }
    public int getTemperature() { return temperature; }
    public String getDescription() { return "Холодное время года"; }
}
```

Файл ClothingSize.java

```
package ru.mirea;

public enum ClothingSize {
    XXS(32) {
        @Override
        public String getDescription() {
            return "Детский размер";
        }
    },
    XS(34), S(36), M(38), L(40);

    private int euroSize;

    ClothingSize(int euroSize) {
        this.euroSize = euroSize;
    }

    public int getEuroSize() { return euroSize; }
    public String getDescription() { return "Взрослый размер"; }
}
```

Файл MenClothing.java

```
package ru.mirea;
```

```
public interface MenClothing {
    void dressMan();
}
```

Файл WomenClothing.java

```
package ru.mirea;

public interface WomenClothing {
    void dressWomen();
}
```

Файл Clothes.java

```
package ru.mirea;

public abstract class Clothes {
    protected ClothingSize size;
    protected double price;
    protected String color;
    protected String brand;

    public Clothes(ClothingSize size, double price, String color) {
        this.size = size;
        this.price = price;
        this.color = color;
    }

    public void setBrand(String brand) {
        this.brand = brand;
    }

    @Override
    public String toString() {
        return brand + ", размер " + size + ", цвет " + color
                + ", цена " + (int) price + " руб.";
    }
}
```

Файл TShirt.java

```
package ru.mirea;

public class TShirt extends Clothes implements MenClothing, WomenClothing {
    public TShirt(ClothingSize size, double price, String color) {
        super(size, price, color);
    }

    @Override
    public void dressMan() {
        System.out.println("Мужская футболка:");
        System.out.println(this);
    }

    @Override
    public void dressWomen() {
        System.out.println("Женская футболка:");
        System.out.println(this);
    }
}
```

Файл Pants.java

```
package ru.mirea;

public class Pants extends Clothes implements MenClothing, WomenClothing {
    public Pants(ClothingSize size, double price, String color) {
        super(size, price, color);
    }

    @Override
    public void dressMan() {
        System.out.println("Мужские штаны:");
        System.out.println(this);
    }

    @Override
    public void dressWomen() {
        System.out.println("Женские штаны:");
        System.out.println(this);
    }
}
```

Файл Skirt.java

```
package ru.mirea;

public class Skirt extends Clothes implements WomenClothing {
    public Skirt(ClothingSize size, double price, String color) {
        super(size, price, color);
    }

    @Override
    public void dressWomen() {
        System.out.println("Юбка:");
        System.out.println(this);
    }
}
```

Файл Tie.java

```
package ru.mirea;

public class Tie extends Clothes implements MenClothing {
    public Tie(ClothingSize size, double price, String color) {
        super(size, price, color);
    }

    @Override
    public void dressMan() {
        System.out.println("Галстук:");
        System.out.println(this);
    }
}
```

Файл Atelier.java

```
package ru.mirea;

public class Atelier {
    public void dressMan(Clothes[] clothes) {
        for (int i = 0; i < clothes.length; i++) {
```

```
            if (clothes[i] instanceof MenClothing) {
                MenClothing item = (MenClothing) clothes[i];
                item.dressMan();
            }
        }
    }

    public void dressWomen(Clothes[] clothes) {
        for (int i = 0; i < clothes.length; i++) {
            if (clothes[i] instanceof WomenClothing) {
                WomenClothing item = (WomenClothing) clothes[i];
                item.dressWomen();
            }
        }
    }
}
```